ENGINEERING REPORT · CI/CD PERFORMANCE · TOUR BACKEND

Split Lanes

How the automated test system went from a ~20-minute wait to a projected 8 minutes — and exactly what that costs, in plain terms.

~8 min	1,604	$0	+48%
PROJECTED WAIT, WAS 21M21S	AUTOMATED CHECKS PER CHANGE	BILLED THIS MONTH, EVERY DAY	MORE RAW COMPUTE TIME FOR THAT SPEED

PREPARED FOR
NativeAmericanTours Engineering

PREPARED BY
Devat Karetha — Solutions Lead

DATE
September 25, 2026

STATUS
Split Lanes is live in production CI

01 · THE PROBLEM

Every code change waited in line

Before a developer's change can be approved and shipped, the system automatically re-checks the entire product — to make sure nothing broke. That check used to take about 20 minutes, every single time.

Whenever an engineer proposes a change — fixing a bug, adding a feature — GitHub automatically spins up a fresh, disposable copy of the booking platform and runs **1,604 individual checks** against it (6,088 individual assertions in total). Each check asks a very specific question, for example:

Does a partial refund add up?

"Refund removed pax addons for rate group refunds matching pax items" — checks that removing one traveler from a group booking refunds exactly the right amount.

Does a customer's email get accepted?

"Customer email validation" — checks dozens of real-world email formats so a legitimate booking is never rejected at checkout.

Does a disputed charge notify the right people?

"Dispute notification service" — checks that a bank chargeback alerts the operator before the response deadline passes.

Does the money end up in the right account?

"Stripe application-fee rounding" — checks that a payment split across the platform and a tour operator never loses a cent to rounding.

An engineer opens a change, then waits. In practice that means either sitting idle for ~20 minutes, or context-switching to something else and losing focus — and across a whole team making many changes a week, that wait adds up fast.



How every code change was checked before today: one computer, one line, ~20 minutes end to end.

02 · QUICK WINS — ALREADY SHIPPED

Two fixes, before touching parallelism

Before running checks side-by-side, we found two things making every single check slower than it needed to be — both fixed with no new risk.

01 A needless restart, every single check

For every one of the 1,604 checks, the system was fully restarting itself from scratch — not once, but **twice** — even though only about a fifth of the checks actually needed that second restart. It's the equivalent of closing and reopening an entire application twice, just to glance at one setting.

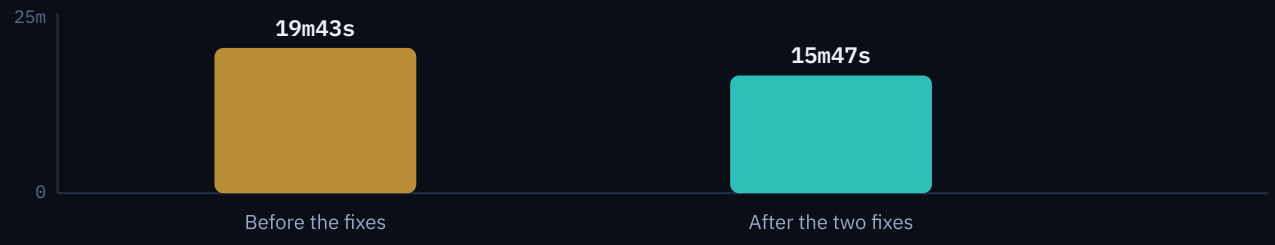
`Fixed — one restart, only where needed`

02 Re-downloading the same ingredients every time

The system was also re-fetching all ~167 external code packages it depends on, from the internet, from scratch, on every single run — instead of remembering what it fetched last time and reusing it.

`Fixed — packages are now cached between runs`

Result: measured, not estimated



Measured, single-lane wait time, from the actual GitHub Actions run history — before parallel lanes were introduced.

03 · THE BIG DECISION

Instead of one line, open more lanes

The two fixes made each check faster. The next lever is running many checks at the same time — like opening more checkout lanes at a store instead of one register for everyone. There are two different ways to do that.

A · SPLIT LANES — four separate computers



no connections between lanes — nothing shared

B · SHARED LANES — one computer, four workers inside



The real difference between the two approaches: how many computers are set up, and whether test data is ever in the same place twice.

A · Split Lanes

LIVE NOW

The 1,604 checks are split into 4 equal batches; each batch gets its own completely separate computer, with its own separate practice database.

Safety: zero chance one lane's data touches another's — by construction. **Cost:** setup is paid 4 times instead of once.

B · Shared Lanes

NOT BUILT YET

One computer runs 4 workers side by side, each working through its own slice of checks, inside the same environment.

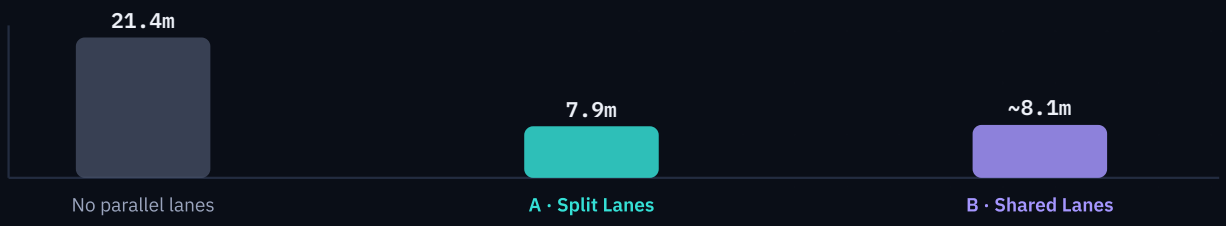
Cost: only one computer to set up — cheaper. **Safety:** needs new engineering to guarantee workers never collide.

04 · THE TRADE-OFF

Same speed, very different bill

Here is the part that matters for planning: both approaches finish at roughly the same speed. They differ enormously in how much computer time that speed actually uses.

Minutes an engineer waits, per code change



Computer-minutes billed, per code change



Split Lanes and Shared Lanes finish at nearly the same time — but Split Lanes uses roughly 4x more raw computer time, because each of its 4 lanes sets itself up independently.

	A · SPLIT LANES	B · SHARED LANES
Engineer's wait, per change	~7.9 minutes	~8.1 minutes
Computer-minutes billed	~31.6 minutes	~8.1 minutes
Chance of test data colliding	None — impossible by design	Real — needs new safeguards
Engineering work to build	None — already shipped	Meaningful — new isolation logic
Status today	Live in production	Documented option, not started

05 · WHAT IT COSTS

A monthly allowance, not a per-minute bill

GitHub, who runs this automated checking system for us, works like a mobile phone data plan.

> **The plan analogy:** the company's GitHub plan already includes a set number of computer-minutes every month, as part of what we pay today. Only minutes used **above** that included amount cost anything extra — the same way going over your phone's data cap is the only time it charges more.

This month's actual usage



Reconstructed from the org's own September 1–25 billing report: usage that would cost ~$11 at standard rates is fully covered by the plan's included allowance, so the actual bill stays at $0.

Nothing has actually been charged, this month or any prior day

Every day shown above billed $0, even as raw usage climbed — the plan's monthly allowance absorbed all of it.

What turning on Split Lanes adds

Split Lanes is projected to add roughly **850 extra computer-minutes a month** — about **$5** of the kind of usage shown above, using this account's own observed rate. That is small next to the pattern above, and very likely to stay fully covered the same way — worth a quick double-check in `Settings → Billing → Overview` before scaling this further (for example, going from 4 lanes to 8).

Devat Karetha — Solutions Lead

06 · WHERE THINGS STAND

What changed, and what's optional next

1. **Live today: Split Lanes**
   Turned on in production CI. Typical wait per code change dropped from ~20 minutes to a projected ~8 minutes.

2. **Costs almost nothing extra, right now**
   The additional computer time used is small next to our monthly allowance, and every day this month has billed $0.

3. **One option on the shelf: Shared Lanes**
   Could eventually cut the extra computer-minutes to nearly zero, but needs dedicated engineering time to build the safety work first. Not recommended to start until there's a clear reason to prioritize it.

A few terms, plainly

Pull Request
A proposed code change, submitted for review before it ships.

Automated test
A small program that checks one specific behavior still works correctly.

CI/CD pipeline
The automated process that builds, tests, and prepares a change to ship.

Container
A disposable, self-contained mini-computer spun up just for one job.

Test database
A throwaway practice copy of real data, safe to write to and destroy.

Sharding / lane
Splitting one big job into smaller, independent batches run side by side.

Devat Karetha
Solutions Lead · NativeAmericanTours · September 25, 2026